

PROYECTO INTEGRADOR — BIG DATA

Documentación de la Arquitectura y del Modelo de Datos Ingesta, Limpieza y Enriquecimiento de Datos de Mercado Cripto

Estudiante: Jonatan Dair Ávila Agamez

Programa: Ingeniería de Software y Datos — IU Digital de Antioquia

Curso: Big Data — Proyecto Integrador (EA4)

Repositorio: github.com/JonathanAgamez25/Ingesta_Api

Documento entregado en la carpeta /docs del repositorio

1. Introducción y descripción global de la arquitectura

Este documento describe la arquitectura del proyecto integrador de Big Data desarrollado a lo largo de las actividades EA1, EA2 y EA3, y define el modelo de datos que resulta de integrar sus tres etapas. El proyecto simula, sobre un entorno local automatizado con GitHub Actions, un pipeline de datos de tipo **ELT por lotes (batch)** que extrae información pública de mercado de criptomonedas, la almacena en una base de datos analítica, la depura y finalmente la enriquece con fuentes externas heterogéneas.

La arquitectura se organiza en tres fases secuenciales que corresponden exactamente a las tres actividades previas del proyecto:

1. **Ingesta (EA1 — `src/ingestion.py`):** extrae un snapshot de las 100 criptomonedas de mayor capitalización de mercado desde el API público de CoinGecko y lo persiste en una base de datos SQLite.
2. **Preprocesamiento y limpieza (EA2 — `src/cleaning.py`):** lee el histórico acumulado en esa base de datos, diagnostica su calidad, corrige tipos de dato y nulos, y genera dos variables derivadas.
3. **Enriquecimiento (EA3 — `src/enrichement.py`):** integra el dataset limpio con seis fuentes externas en formatos distintos (JSON, XLSX, CSV, XML, HTML y TXT), añadiendo ocho columnas de contexto adicional por cada moneda.

Las tres etapas están encadenadas en un único flujo de trabajo de GitHub Actions (`.github/workflows/bigdata.yml`), que actúa como el mecanismo de orquestación y automatización del pipeline completo, y que además cumple el rol de "entorno de nube simulado" exigido por el enunciado de la actividad: cada corrida del workflow reproduce, en un runner efímero de GitHub, el mismo ciclo extracción → almacenamiento → transformación → publicación de evidencias que tendría un pipeline productivo en un proveedor cloud real.

2. Componentes principales

Base de datos analítica SQLite (<code>src/db/ingestion.db</code>), con dos tablas: <code>ingestion_runs</code> y <code>coins_market</code> . Funciona como el "almacenamiento en la nube simulado" que conecta EA1 con EA2.	Script de ingesta <code>ingestion.py</code> : consume el endpoint <code>/coins/markets</code> de CoinGecko, crea el esquema si no existe, inserta el snapshot y genera evidencia en Excel y un reporte de auditoría comparando API vs. base de datos.
Script de limpieza <code>cleaning.py</code> : lee el histórico completo de <code>coins_market</code> (sin volver a llamar al API), diagnostica calidad, limpia y transforma, y deja un reporte de auditoría antes/después.	Script de enriquecimiento <code>enrichement.py</code> : cruza el dataset limpio contra seis fuentes externas por la clave <code>id</code> mediante <i>left join</i>, y deja un reporte de auditoría por fuente.
Fuentes de enriquecimiento Seis archivos en <code>src/enrichment_sources/</code> : <code>categorias.json</code> , <code>consenso.xlsx</code> , <code>sitios_web.csv</code> , <code>casos_uso.xml</code> , <code>tipo_red.html</code> y <code>notas.txt</code> .	Mecanismo de automatización GitHub Actions (<code>bigdata.yml</code>): ejecuta las tres etapas en secuencia, verifica sus artefactos, los publica como artefacto descargable y confirma las evidencias actualizadas al repositorio mediante <code>git-auto-commit-action</code> .

3. Diagramas de arquitectura

3.1 Flujo de datos general (de la extracción a la publicación de evidencias)

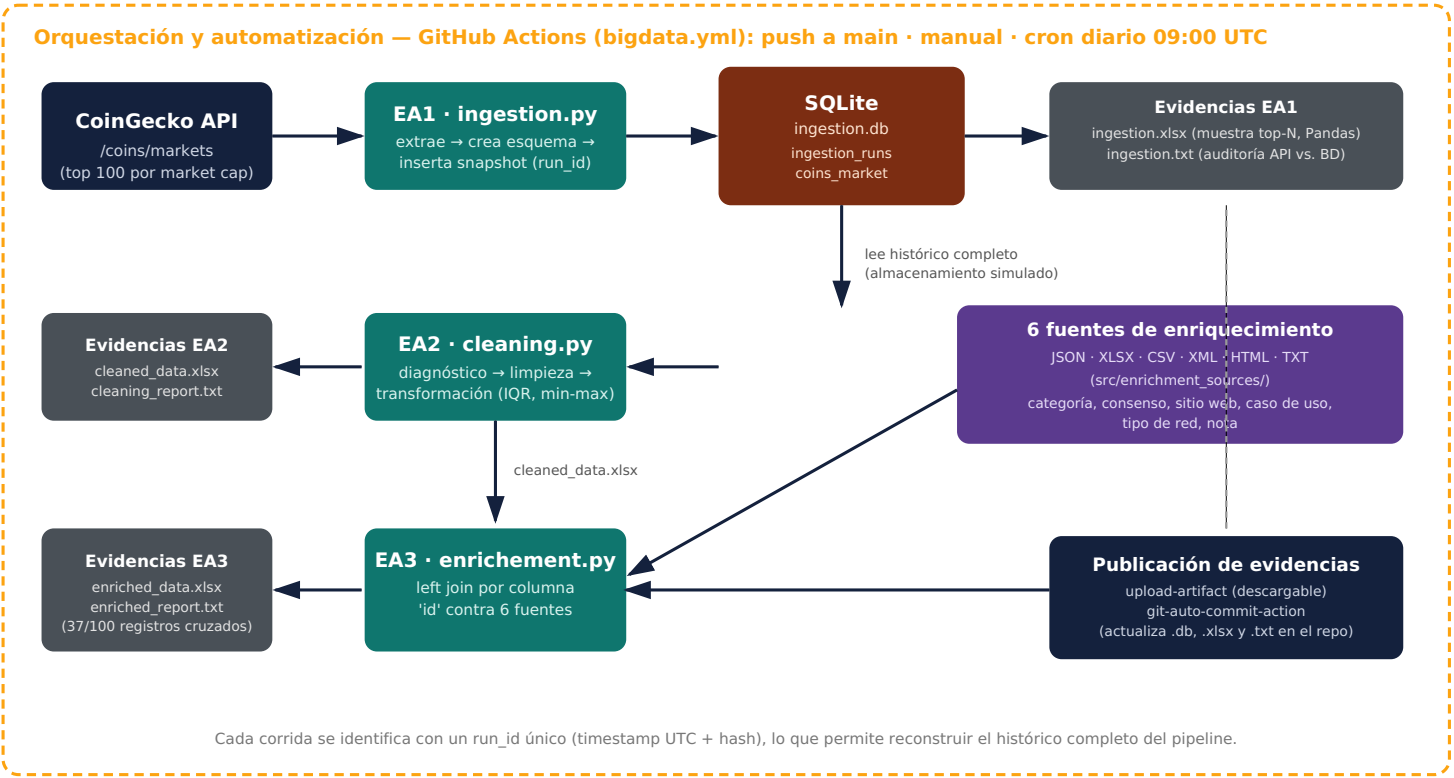


Figura 1. Flujo de datos desde la extracción del API hasta la publicación de evidencias, orquestado por el workflow de GitHub Actions.

3.2 Detalle del proceso de preprocesamiento y enriquecimiento



Cada etapa genera una muestra en Excel y un reporte de auditoría en texto plano que documenta la trazabilidad del proceso.

Figura 2. Detalle interno de las etapas de preprocesamiento (EA2) y enriquecimiento (EA3).

4. Modelo de datos

4.1 Esquema resultante de la integración

La base de datos analítica (`src/db/ingestion.db`) contiene dos tablas base, creadas por `ingestion.py` (EA1). Las etapas EA2 y EA3 no modifican estas tablas: leen desde ellas (o desde su exportación en Excel) y producen datasets derivados en Excel, preservando la base transaccional intacta como fuente de verdad histórica.

Tabla `ingestion_runs` — bitácora de ejecuciones

Campo	Tipo	Restricción	Descripción
run_id	TEXT	PRIMARY KEY	Identificador único de la corrida (timestamp UTC + hash aleatorio, p. ej. <code>20260927T142239Z-bc386a</code>)
executed_at	TEXT	NOT NULL	Fecha y hora de ejecución en formato ISO 8601
source_url	TEXT	NOT NULL	URL del endpoint consultado
records_extracted	INTEGER	NOT NULL	Registros devueltos por el API en esa corrida
records_inserted	INTEGER	NOT NULL	Registros efectivamente insertados en <code>coins_market</code>
status	TEXT	NOT NULL	<code>OK</code> o <code>ERROR: <detalle></code> si la corrida falló

Tabla `coins_market` — snapshot de mercado por corrida

Campo	Tipo	Restricción	Descripción
run_id	TEXT	PK (compuesta) · FK → <code>ingestion_runs(run_id)</code>	Corrida a la que pertenece el registro
coin_id	TEXT	PK (compuesta)	Identificador de la moneda en CoinGecko (renombrado desde <code>id</code> del API)
symbol	TEXT	—	Símbolo bursátil (<code>btc</code> , <code>eth</code> , ...)
name	TEXT	—	Nombre de la criptomoneda
current_price	REAL	—	Precio actual en USD
market_cap	REAL	—	Capitalización de mercado
market_cap_rank	INTEGER	—	Posición en el ranking por capitalización
total_volume	REAL	—	Volumen de negociación en 24h
high_24h / low_24h	REAL	—	Máximo y mínimo de precio en 24h
price_change_24h	REAL	—	Variación absoluta de precio en 24h
price_change_percentage_24h	REAL	—	Variación porcentual de precio en 24h
circulating_supply / total_supply	REAL	—	Oferta circulante y total del activo
ath / ath_date	REAL / TEXT	—	Precio histórico más alto (all-time high) y su fecha
last_updated	TEXT	—	Fecha de la última actualización reportada por el API

Sobre este esquema base, EA2 añade dos columnas derivadas al dataset limpio (no a la tabla SQLite, sino al DataFrame exportado en `cleaned_data.xlsx`): `is_price_change_outlier` (booleano, método IQR) y `market_cap_normalized` (real, escalado min-max por `run_id`). EA3 añade ocho columnas más de contexto, provenientes de las seis fuentes externas: `categoria`, `subcategoria`, `mecanismo_consenso`, `anio_lanzamiento`, `sitio_web`, `caso_de_uso`, `tipo_red` y `nota`.

4.2 Diagrama entidad-relación

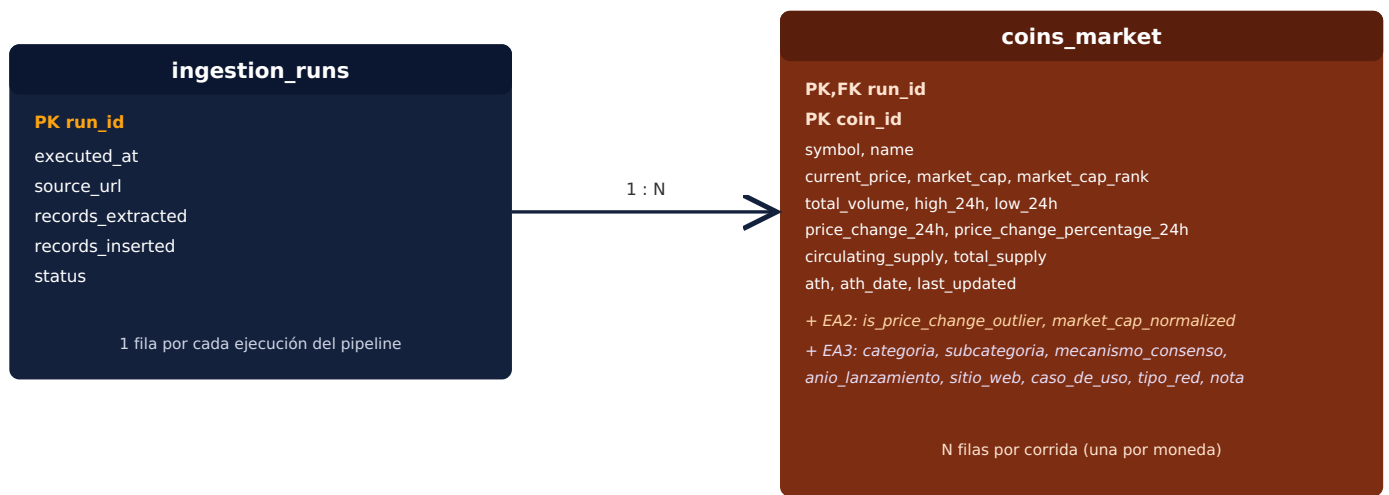


Figura 3. Diagrama entidad-relación. Una corrida (`ingestion_runs`) contiene N registros de mercado (`coins_market`); las columnas de EA2 y EA3 se agregan sobre el dataset derivado, no sobre la tabla transaccional.

4.3 Justificación del modelo de datos

El modelo se diseñó como una **tabla de bitácora** (`ingestion_runs`) separada de una **tabla de hechos por snapshot** (`coins_market`), en lugar de una única tabla plana, por tres razones concretas:

- **Trazabilidad del proceso:** al registrar cada corrida (cuándo se ejecutó, cuántos registros trajo el API, cuántos se insertaron y con qué estado), es posible auditar el pipeline en cualquier momento sin depender de los logs efímeros del runner de GitHub Actions.
- **Serie de tiempo implícita:** como cada fila de `coins_market` conserva su `run_id` y el workflow corre diariamente vía `cron`, la tabla se convierte por sí sola en un histórico de precios por moneda a lo largo del tiempo, sin necesidad de una tabla adicional de series temporales. La base actual acumula 34 corridas y 3.400 filas históricas (100 monedas × 34 días).
- **Clave compuesta natural:** (`run_id`, `coin_id`) como llave primaria de `coins_market` evita duplicados dentro de una misma corrida y hace explícita, sin columnas adicionales, la relación uno-a-muchos entre una ejecución y las monedas que capturó.

Este diseño facilita directamente las etapas posteriores: EA2 puede leer el histórico completo con un único `SELECT * FROM coins_market` y calcular estadísticos por `run_id` (como la normalización min-max), y EA3 puede enriquecer el dataset por la clave `id / coin_id` sin preocuparse por a qué corrida pertenece cada fila, porque los atributos de contexto (categoría, mecanismo de consenso, etc.) son propiedades del activo y no de la corrida.

5. Justificación de herramientas y tecnologías

Herramienta	Rol en el proyecto	Por qué se eligió
SQLite	Base de datos analítica embebida (<code>ingestion.db</code>)	No requiere servidor ni instalación adicional, es compatible de forma nativa con Python (<code>sqlite3</code>) y con GitHub Actions al ser un único archivo versionable como artefacto. Para el volumen del proyecto (cientos de filas por corrida) ofrece consultas SQL completas sin el costo operativo de un motor cliente-servidor.
Pandas	Lectura/escritura de la base de datos, limpieza, transformación, cruces (<i>merge</i>) y exportación a Excel	Su integración directa con <code>sqlite3</code> (<code>read_sql_query</code>) y con múltiples formatos de archivo (Excel, CSV, XML) permitió resolver en un solo framework las tres etapas —ingesta, limpieza y enriquecimiento— sin herramientas adicionales por formato.
GitHub Actions	Orquestación y automatización del pipeline completo	Ejecuta las tres etapas en secuencia sobre un runner limpio (simulando así un entorno de nube efímero), programa corridas diarias con <code>schedule: cron</code> , verifica los artefactos de cada etapa antes de continuar, y publica la evidencia mediante <code>upload-artifact</code> y <code>git-auto-commit-action</code> — sin costo adicional para un repositorio público.
PySpark	No utilizado en la implementación actual	Con snapshots de 100 filas por corrida (miles de filas en el histórico completo), Pandas procesa el volumen actual sin cuellos de botella, por lo que introducir Spark habría añadido complejidad de despliegue sin un beneficio real. Se documenta como ruta de escalamiento en la sección de recomendaciones.

5.1 Simulación del entorno cloud

El entorno de nube se simula combinando dos elementos que en un despliegue real corresponderían a servicios administrados distintos: `ingestion.db` cumple el papel de un almacenamiento persistente (equivalente a un bucket S3, un contenedor de Azure Blob Storage o un volumen de base de datos gestionada), y el workflow de GitHub Actions cumple el papel de un orquestador de trabajos por lotes (equivalente a un cron job en una VM, a Airflow o a un pipeline de Azure Data Factory). Cada corrida del workflow arranca en un runner efímero de Ubuntu, clona el repositorio, reconstruye el entorno de Python desde `setup.py`, ejecuta las tres etapas contra el archivo `ingestion.db` versionado en el repositorio, y al finalizar confirma (*commit*) el estado actualizado de la base de datos y las evidencias — reproduciendo así el ciclo completo de un pipeline batch en la nube sin necesidad de una cuenta en un proveedor cloud real.

6. Flujo de datos y automatización

El flujo completo, tal como está codificado en `.github/workflows/bigdata.yml`, sigue esta secuencia en cada ejecución del workflow:

- Se clona el repositorio y se configura Python 3.11.
- Se instalan las dependencias declaradas en `setup.py` (`requests`, `pandas`, `openpyxl`).
- EA1:** se ejecuta `ingestion.py`, que genera un `run_id` nuevo, consulta el API, crea el esquema si no existe, inserta el snapshot y registra la corrida en `ingestion_runs`; se verifica la existencia de `ingestion.db`, `ingestion.xlsx` e `ingestion.txt`.
- EA2:** se ejecuta `cleaning.py`, que lee el histórico completo (todas las corridas acumuladas), lo limpia y transforma, y exporta `cleaned_data.xlsx` y `cleaning_report.txt`.
- EA3:** se ejecuta `enrichement.py`, que cruza el dataset limpio contra las seis fuentes externas y exporta `enriched_data.xlsx` y `enriched_report.txt`; el workflow imprime este último reporte directamente en el log de la corrida.
- Se verifican explícitamente los artefactos de las tres etapas (fallos en cualquier verificación detienen el workflow).
- Los siete archivos generados se publican como un artefacto descargable de GitHub Actions, identificado con el `run_id` del workflow.
- `git-auto-commit-action` confirma al repositorio el estado actualizado de la base de datos y las evidencias, con el mensaje marcado `[skip ci]` para no disparar una nueva corrida en bucle.

Disparadores del workflow: push a la rama `main`, ejecución manual desde la pestaña Actions, y automáticamente todos los días a las 09:00 UTC — lo que mantiene el histórico de `coins_market` creciendo día a día sin intervención manual.

7. Conclusiones y recomendaciones

Beneficios de la arquitectura propuesta

- El desacople entre ingesta, limpieza y enriquecimiento en tres scripts independientes permite ejecutar, probar y depurar cada etapa por separado sin afectar a las demás.

- El uso de `run_id` como eje transversal del modelo de datos da trazabilidad completa: cualquier fila de `coins_market` puede rastrearse hasta la corrida exacta que la generó.
- Los reportes de auditoría en texto plano en cada etapa cumplen, sin herramientas adicionales, el requisito de documentar la trazabilidad del proceso exigido por la actividad.
- La automatización diaria convierte un proceso que sería manual en una fuente de datos históricos que crece sola, habilitando análisis de tendencia que no serían posibles con una sola extracción puntual.

Limitaciones

- SQLite no soporta escritura concurrente real; es adecuado para el volumen y la frecuencia actuales (una corrida diaria) pero sería un cuello de botella si el pipeline necesitara ejecutarse en paralelo o con mayor frecuencia.
- El cruce de enriquecimiento actualmente coincide en 37 de 100 monedas (37%) porque las seis fuentes externas cubren un catálogo fijo de 50 monedas conocidas, mientras que el snapshot de CoinGecko cambia de ranking día a día; esto deja columnas nulas en las monedas no cubiertas, sin perder el registro base (left join).
- Los artefactos de evidencia (Excel, TXT) no están versionados con un esquema explícito; un cambio en las columnas de origen del API podría romper silenciosamente los reportes si no se valida antes de leerlos.

Recomendaciones para un entorno real de nube

- Migrar `ingestion.db` a una base de datos gestionada (PostgreSQL en RDS/Cloud SQL, o un almacén columnar como BigQuery) para soportar escritura concurrente y consultas analíticas a mayor escala.
- Introducir PySpark (o Spark estructurado en modo local) en la etapa de limpieza cuando el histórico acumulado supere el rango en que Pandas deja de ser eficiente en memoria, aprovechando que la lógica de limpieza ya está expresada en operaciones vectorizadas fácilmente portables a Spark DataFrames.
- Reemplazar el almacenamiento de archivos del repositorio por un bucket de objetos (S3 / Blob Storage) para la base de datos y las evidencias, y usar GitHub Actions únicamente como orquestador, no como almacenamiento.
- Ampliar las fuentes de enriquecimiento a un catálogo más amplio y actualizado, para elevar el porcentaje de coincidencia por encima del 37% actual.